

DP Computer Science: A 3.3.6 Database Transactions (HL Only)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Describe the role of **Atomicity, Consistency, Isolation, and Durability (ACID)** in ensuring the reliable processing of database transactions
- Explain how each ACID property contributes to maintaining **data integrity** in a database
- Describe the function of the **Transaction Control Language (TCL)** commands: `BEGIN TRANSACTION` , `COMMIT` , and `ROLLBACK`
- Explain how ACID properties and TCL commands work together to ensure data integrity

Key Vocabulary

Term	Definition
Transaction	A sequence of one or more SQL operations that are treated as a single logical unit of work
Data integrity	The accuracy, consistency, and reliability of data in a database
ACID	Atomicity, Consistency, Isolation, Durability—the four properties that ensure reliable transaction processing
Atomicity	A transaction is treated as a single, indivisible unit—either all operations are completed, or none are
Consistency	A transaction brings the database from one valid state to another, preserving all rules and constraints
Isolation	Concurrent transactions do not interfere with each other; each appears to run in isolation
Durability	Once a transaction is committed, its changes are permanent, even in the event of a system failure
COMMIT	Permanently saves all changes made during a transaction
ROLLBACK	Undoes all changes made during a transaction, returning to the previous state
BEGIN TRANSACTION	Marks the starting point of a transaction

Relevant ATL Skills

Skill	Description
Thinking	Understanding and applying concepts; analysing ACID properties; evaluating importance of transaction management
Communication	Discussing ACID concepts using appropriate terminology; explaining TCL commands

Skill	Description
Self-Management	Organising understanding of ACID and TCL; managing time

2. What is a Database Transaction?

Definition

A transaction is a sequence of one or more SQL operations that are treated as a single logical unit of work. Either all operations in the transaction are completed successfully, or none of them are.

Why Do We Need Transactions?

text

WHY DO WE NEED TRANSACTIONS?

Scenario: Transferring money between bank accounts

Step 1: UPDATE AccountA SET Balance = Balance - 100

Step 2: UPDATE AccountB SET Balance = Balance + 100

What could go wrong?

⚠️ Power failure after Step 1:

└ Money is deducted from Account A but never added to B

└ The money is lost!

⚠️ System crash during Step 2:

└ Data becomes inconsistent

└ Database integrity is compromised

✅ A transaction ensures either BOTH steps happen,

✅ or NEITHER step happens—no partial updates!

Real-World Examples of Transactions

Scenario	Operations	Why a Transaction is Needed
Bank Transfer	Withdraw from Account A; Deposit to Account B	Money must not be lost; both must happen or neither
Online Order	Create order; Reduce stock; Charge customer; Send confirmation	All must succeed; otherwise customer is charged without product
Booking System	Reserve seat; Charge card; Send confirmation	Seat must not be double-booked; payment must be processed
Hospital System	Update patient records; Log change; Send alert	Critical data must be consistent; changes must be tracked

3. The ACID Properties

What is ACID?

ACID is an acronym representing the four properties that ensure reliable processing of database transactions. These properties work together to maintain data integrity.



4. Detailed Explanation of Each ACID Property

Atomicity

Atomicity ensures that a transaction is treated as a single, indivisible unit. Either all operations in the transaction are completed successfully, or none of them are.

Aspect	Description
Principle	"All or nothing"
What it Prevents	Partial updates; data being left in an inconsistent state
How it Works	The database tracks all changes made during a transaction. If the transaction fails, all changes are undone (rolled back).
Example	Bank transfer: both debit and credit operations succeed, or neither does

Analogy: Atomicity is like a **light switch**—it's either on or off. There's no "half-on" state.

Atomicity Example

text

ATOMICITY EXAMPLE

Transaction: Transfer £100 from Account A to Account B

BEGIN TRANSACTION

Step 1: UPDATE AccountA SET Balance = Balance - 100

Step 2: UPDATE AccountB SET Balance = Balance + 100

If both succeed: COMMIT

If either fails: ROLLBACK

✔ If Step 1 fails → ROLLBACK → No changes made

✔ If Step 2 fails → ROLLBACK → Both steps are undone

✔ If both succeed → COMMIT → Changes are permanent

Consistency

Consistency ensures that a transaction brings the database from one valid state to another, preserving all defined rules, constraints, and relationships.

Aspect	Description
Principle	"Valid state to valid state"
What it Prevents	Data that violates constraints (foreign keys, unique, check constraints)
How it Works	The database checks all constraints after each transaction. If a constraint is violated, the transaction is rolled back.
Example	If a foreign key references a non-existent record, the transaction fails

Analogy: Consistency is like **recipe measurements**—if a recipe calls for 2 cups of flour and you add only 1.5 cups, the result won't work (inconsistent state).

Consistency Example

```
sql

-- Suppose there is a foreign key constraint
-- Order.CustomerID must exist in Customer.CustomerID

BEGIN TRANSACTION;

-- This will violate the foreign key constraint because
-- Customer 999 does not exist
INSERT INTO Orders (OrderID, CustomerID, Total)
VALUES (1001, 999, 50.00);

-- The database will reject this and roll back the transaction
-- Consistency is preserved!
```

Isolation

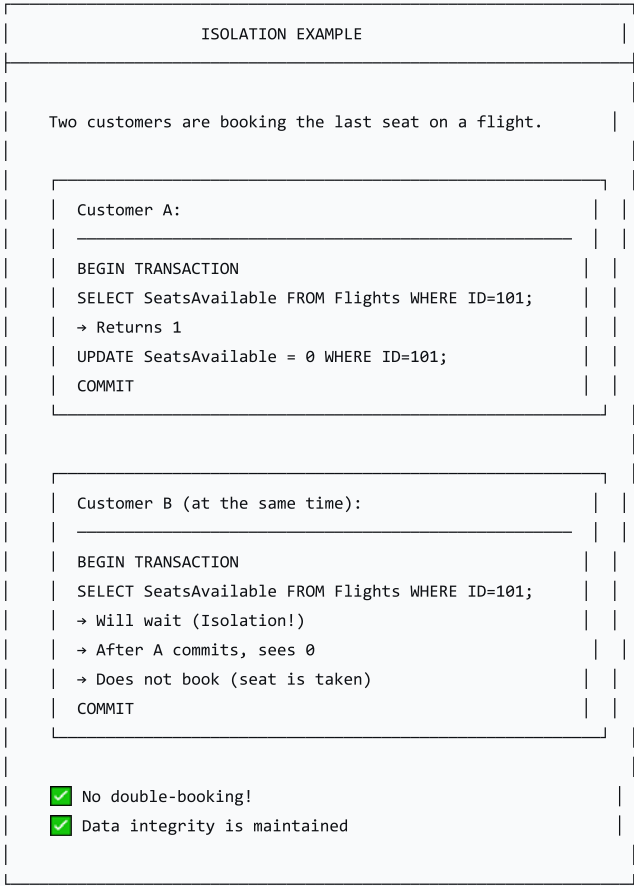
Isolation ensures that concurrent transactions do not interfere with each other. Each transaction appears to run in isolation, as if it were the only transaction being processed.

Aspect	Description
Principle	"No interference"
What it Prevents	Dirty reads, non-repeatable reads, phantom reads
How it Works	Transactions are serialized or use locking mechanisms to prevent conflicts
Example	Two users simultaneously booking the last seat on a flight—only one should succeed

Analogy: Isolation is like **queueing**—even if multiple people are at the bank, each transaction is processed one at a time at each teller.

Isolation Example

text



Durability

Durability ensures that once a transaction is committed, its changes are permanent, even in the event of a system failure.

Aspect	Description
Principle	"Permanent changes"
What it Prevents	Loss of committed changes due to system crashes or failures

Aspect	Description
How it Works	Changes are written to persistent storage (hard drive) before the transaction is considered complete
Example	After confirming a bank transfer, the money remains in the account even if the system restarts

Analogy: Durability is like **writing in ink**—once it's on paper, it doesn't wash away.

Durability Example

text

DURABILITY EXAMPLE

BEGIN TRANSACTION
UPDATE AccountB SET Balance = Balance + 100
COMMIT

 Power failure IMMEDIATELY after COMMIT

What happens?

 Changes are saved to disk BEFORE COMMIT completes

 When the system restarts, the transaction IS reflected in the database

 The change is NOT lost!

 Committed transactions survive system failures

5. ACID Properties Summary

Property	Description	Key Idea	Violation Example
Atomicity	All or nothing	Transaction is indivisible	Partial updates (money deducted but not added)
Consistency	Valid state to valid state	Preserve rules and constraints	Foreign key violation
Isolation	No interference	Transactions run independently	Double-booking a seat
Durability	Permanent changes	Committed changes survive failures	Lost transactions after power failure

6. Transaction Control Language (TCL)

What is TCL?

TCL (Transaction Control Language) is a subset of SQL used to manage database transactions. It allows programmers to control when changes are made permanent and when they are undone.

TCL Commands

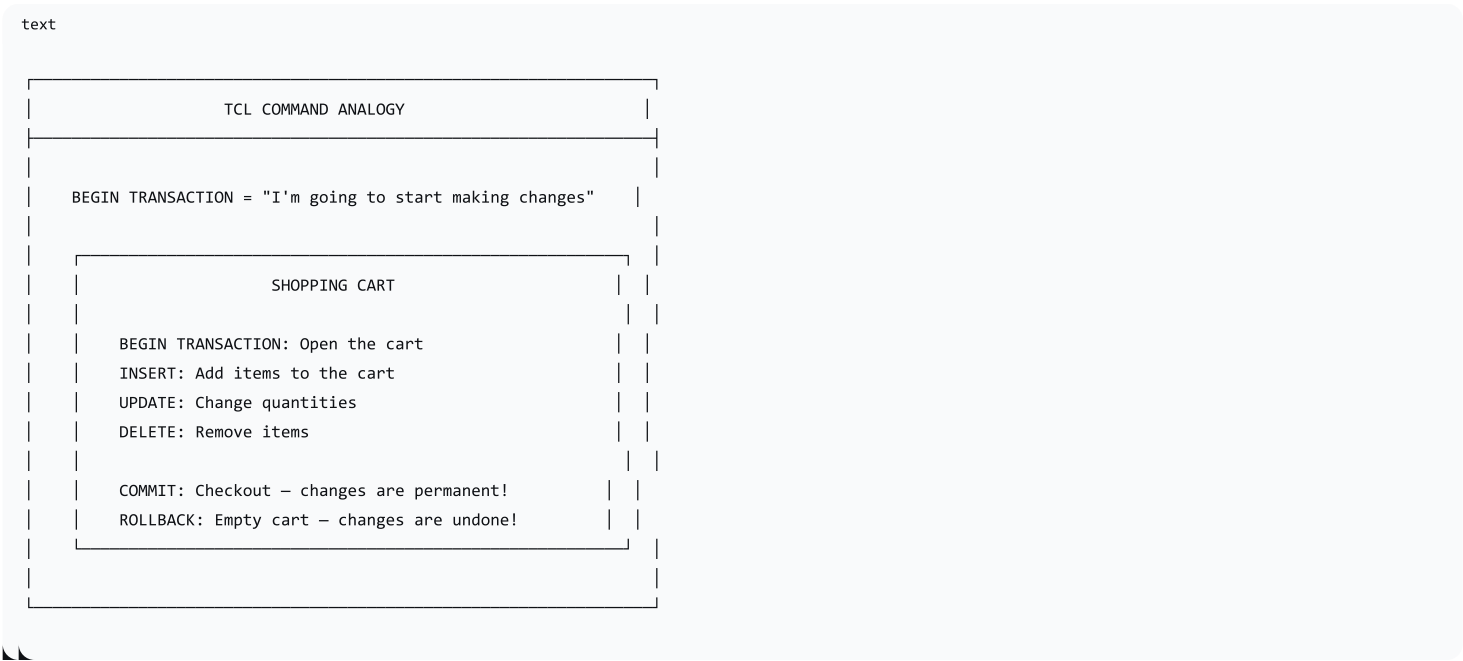
Command	Purpose	Effect
BEGIN TRANSACTION	Start a transaction	Marks the beginning of a transaction
COMMIT	Save changes permanently	All changes become permanent; transaction ends
ROLLBACK	Undo changes	All changes are undone; transaction ends

Syntax

```
sql

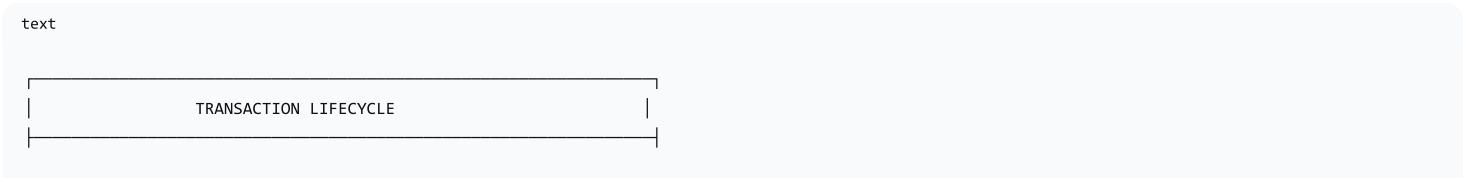
BEGIN TRANSACTION;
-- SQL operations (INSERT, UPDATE, DELETE)
-- ...
COMMIT; -- or ROLLBACK;
```

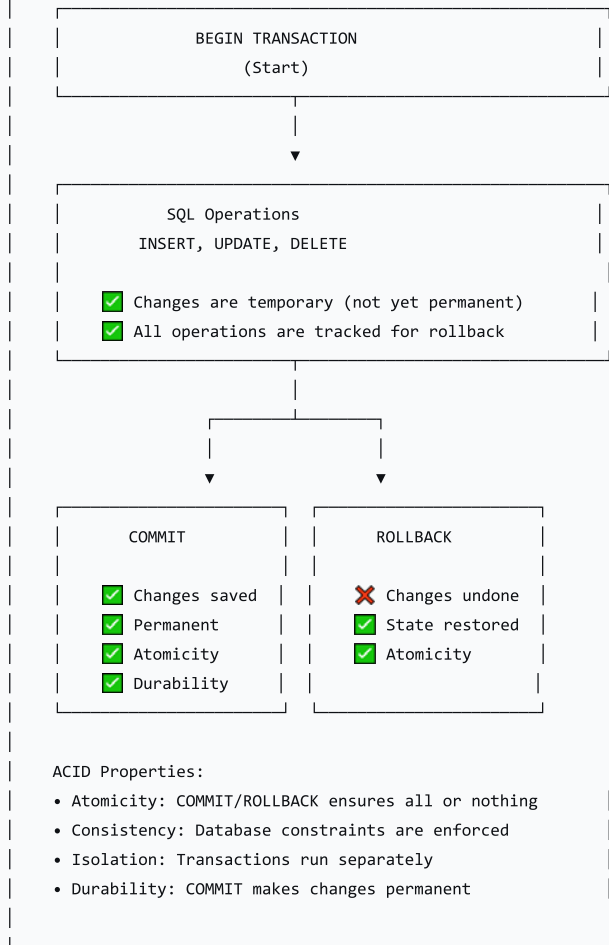
TCL Command Analogy



7. How TCL Commands Support ACID

Transaction Lifecycle





Mapping ACID to TCL

ACID Property	How TCL Supports It
Atomicity	<code>COMMIT</code> finalises all operations; <code>ROLLBACK</code> undoes all operations—no partial updates
Consistency	The database checks constraints before allowing <code>COMMIT</code>
Isolation	The database manages locks to prevent transaction interference
Durability	<code>COMMIT</code> writes changes to permanent storage before completing

8. Practical Example

Scenario: Bank Transfer

```
sql

-- Start the transaction
BEGIN TRANSACTION;

-- Step 1: Deduct £100 from Account A
UPDATE Accounts
SET Balance = Balance - 100
```



```
WHERE AccountID = 1;

-- Step 2: Add £100 to Account B
UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountID = 2;

-- Check if both operations were successful
-- If yes, make the changes permanent
COMMIT;

-- If there was an error, undo all changes
-- ROLLBACK;
```

What Happens at Each Step

Step	State	Explanation
BEGIN	Transaction started	Database prepares for a transaction
UPDATE A	Balance temporarily reduced	Changes are not yet permanent
UPDATE B	Balance temporarily increased	Changes are not yet permanent
COMMIT	Both changes become permanent	Atomicity: both steps are saved
ROLLBACK (if needed)	Both changes are undone	Atomicity: neither step is saved

9. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Need for reliable data management; potential problems without transactions
Data Integrity	5 min	Definition of data integrity; role of transactions
ACID Properties	15 min	Introduction to ACID; detailed explanation of Atomicity with examples
ACID (Continued)	10 min	Consistency and Isolation with scenarios
Durability & TCL	10 min	Durability; Introduction to TCL; BEGIN, COMMIT, ROLLBACK
ACID & TCL in Action	5 min	SQL examples demonstrating ACID with TCL commands
Summary & Worksheet	10 min	Recap; introduce worksheet activities; Q&A

10. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of ACID and TCL
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What does the "A" in ACID stand for? Explain it briefly.
2. What is the purpose of the COMMIT command?
3. What happens if a transaction fails after an UPDATE operation?
4. Explain the difference between COMMIT and ROLLBACK.

11. Differentiation

Level	Strategy
Support	Use real-world analogies; visual aids; work through examples together
Challenge	Explore multiple-table transactions; research transaction isolation levels; investigate error handling in different DBMS

12. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.3.6 with examples)
- **Worksheet:** ACID and TCL exercises
- **Worksheet Answers**
- **DB Browser for SQLite** (for practical demonstration)

13. Summary: Key Takeaways

Concept	Key Point
Transaction	A logical unit of work—all or nothing
Atomicity	Either all operations succeed or none do
Consistency	Database must be in a valid state before and after
Isolation	Transactions do not interfere with each other
Durability	Committed changes are permanent
BEGIN TRANSACTION	Marks the start of a transaction
COMMIT	Saves all changes permanently
ROLLBACK	Undoes all changes

ACID — The Big Picture

text

ACID IN SUMMARY	
ATOMICITY (All or nothing)	"All or nothing" – No partial updates Ensure all steps succeed or none do ✔ TCL: COMMIT / ROLLBACK
CONSISTENCY (Rules enforced)	"Valid state to valid state" Preserve constraints and rules ✔ Database checks constraints
ISOLATION (Separate)	"No interference" Transactions run independently ✔ Locking / Serialization
DURABILITY (Permanent)	"Permanent" – Survives system failures ✔ COMMIT writes to persistent storage

"ACID properties are the foundation of reliable database systems—they ensure that your data remains accurate, consistent, and trustworthy."